

Computer Security Laboratory Assignment

Controlled DoS and DDoS Simulation on a Local Web Server

An Application-Layer Availability and Resource Exhaustion Demonstration

Coursework Submission

Department of Computer Science & Engineering

September 24, 2026

Abstract

Denial of Service (DoS) and Distributed Denial of Service (DDoS) attacks aim to diminish the availability of a service by exhausting finite resources required by legitimate users. These critical resources include worker threads, memory buffers, CPU time, and application concurrency slots. This laboratory experiment presents a safe, local, and reproducible demonstration implemented in Python using a multithreaded Flask HTTP server and controlled traffic generators. A web browser acts as the legitimate user client. Under baseline conditions, the user interacts smoothly with the web interface and executes health checks. During an attack scenario, the traffic generator dispatches rapid requests to an expensive workload endpoint (`/work`) until all active worker slots ($C = 4$) are occupied. Subsequent legitimate browser probes to `/health` encounter immediate HTTP 503 Service Unavailable responses. Upon stopping the attack stream, the server promptly recovers to normal operating capacity. Quantitative metrics, interactive browser UI screenshots, and defensive countermeasures are presented.

Contents

1	Introduction	3
1.1	Objectives	3
2	DoS and DDoS Concepts	3
3	System Design	4
3.1	Endpoint Separation Logic	4
4	Server Implementation	5
4.1	Socket Setup and Local Binding	5
4.2	Worker Reservation and Overload Handling	5
5	Traffic Generator Implementation	5
5.1	Single-Source DoS Simulation (<code>client.py</code>)	5
5.2	Multi-Bot DDoS Simulation (<code>ddos_sim.py</code>)	5
6	Legitimate Browser Client	6
7	Build and Execution Procedure	6

8 Demonstration and Screenshots	7
8.1 Normal State	7
8.2 Single-Source DoS Run	7
8.3 Legitimate User During Overload	8
8.4 Multi-Bot DDoS Run	8
8.5 Server Saturation and Recovery	9
9 Results and Discussion	10
9.1 Interpretation	10
10 Defensive Countermeasures	10
11 Conclusion	10
A Viva Voce Examination Preparation Guide	12
B Project Source Code Listings	12
B.1 Target Web Server (code/server.py)	12

1 Introduction

Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS) attacks represent fundamental threats to the availability pillar of the CIA (Confidentiality, Integrity, and Availability) security triad. By flooding a target with excessive connection requests or computationally demanding tasks, adversaries exhaust finite hardware or application-layer capacities. The visible outcome is elevated latency, connection timeouts, rejected requests (HTTP status 503), or total service outages.

This assignment implements a safe, controlled classroom demonstration on localhost (127.0.0.1) using Python 3, Flask, and the Requests library. A standard web browser serves as the legitimate client. During an experiment, client generators request an intentionally expensive endpoint (`/work`), saturating the server's worker pool. When a legitimate browser user attempts a health check (`/health`), the server returns HTTP 503 Service Unavailable. When traffic ceases, the server automatically recovers.

1.1 Objectives

The primary objectives of this laboratory assignment are:

1. Implement a multithreaded HTTP dummy server with deliberately constrained concurrency capacity.
2. Model finite resource exhaustion using a bounded semaphore ($C = 4$ worker slots) and simulated processing delay ($T_p = 0.25$ s).
3. Implement a controlled single-source DoS traffic scenario.
4. Implement a simulated multi-process botnet cluster DDoS scenario.
5. Observe the visual impact in real-time from a legitimate web browser client.
6. Record quantitative metrics (request throughput, HTTP 200 vs. 503 counts, latency ranges).
7. Verify instantaneous system recovery after attack traffic ceases.

2 DoS and DDoS Concepts

A Denial of Service (DoS) attack originates from a single logical source or IP address, creating sufficient concurrent traffic to exhaust the target's capacity. A Distributed Denial of Service (DDoS) attack leverages multiple coordinated sources (often termed a botnet), such that their aggregated traffic swamps capacity while circumventing simple single-IP rate-limiting thresholds.

Table 1: Conceptual comparison between the implemented DoS and DDoS scenarios

Property	DoS Mode	DDoS Mode
Logical Sources	One source	Four independent worker processes
Parallel Activity	Multiple rapid requests from single host	Concurrent multi-process bot cluster
Client Identity	<code>dos-source</code>	<code>bot-1</code> through <code>bot-4</code>
Main Idea	One client floods requests rapidly	Multiple distributed bots generate combined load
Visible Result	Server worker exhaustion → HTTP 503	Faster saturation and higher HTTP 503 drop rate

Local Simulation Boundary Note: Because all components run on a single machine, the DDoS workers share the loopback IP 127.0.0.1. They are simulated virtual bot entities created via separate Python `multiprocessing` processes. This demonstrates application-level distributed demand safely without creating dangerous network floods.

3 System Design

The laboratory testbed consists of four modular components:

Table 2: Components of the local availability testbed

Component	Responsibility
Flask Server (<code>server.py</code>)	Listens on 127.0.0.1:5000, serves the website, processes <code>/health</code> and <code>/work</code> queries, bounds active worker slots, and returns HTTP 503 during overload.
Traffic Generator (<code>client.py</code>)	Executes single-source DoS traffic floods against <code>/work</code> with minimal inter-request delay.
DDoS Cluster (<code>ddos_sim.py</code>)	Spawns independent multiprocessing bot processes to simulate multi-source distributed load.
Interactive Browser Page	Represents the legitimate user; provides a “Check Server” button to probe <code>/health</code> and display live status (SERVER ONLINE / SERVER OVERLOADED).

3.1 Endpoint Separation Logic

- GET `/`: Serves the interactive user dashboard.
- GET `/health`: Lightweight probe for legitimate users. Checks slot availability and returns immediately (< 10 ms).
- GET `/work`: Resource-heavy endpoint for attack simulation. Simulates expensive computation (0.25 s delay), occupying a semaphore slot.
- GET `/stats`: Exposes accepted, rejected, and active counter metrics in JSON format.

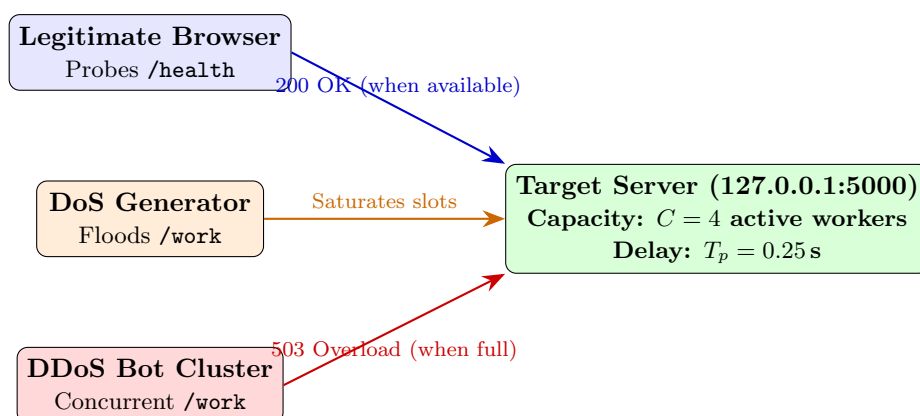


Figure 1: System architecture and request flow during controlled availability simulation.

4 Server Implementation

4.1 Socket Setup and Local Binding

The server is constructed using Python's Flask WSGI framework. To guarantee complete safety, the server explicitly binds only to the IPv4 loopback address 127.0.0.1 on port 5000. This prevents any inbound connections from external devices on the physical local area network.

4.2 Worker Reservation and Overload Handling

Concurrency control is implemented using `threading.BoundedSemaphore(4)`. When a request arrives at `/work` or `/health`, the server executes a non-blocking acquisition attempt:

Listing 1: Thread-safe non-blocking worker slot acquisition

```
1 slots = threading.BoundedSemaphore(4)
2 stats = {"accepted": 0, "rejected": 0}
3 lock = threading.Lock()
4
5 @app.get("/work")
6 def work():
7     # Attempt non-blocking slot reservation
8     acquired = slots.acquire(blocking=False)
9     if not acquired:
10         with lock:
11             stats["rejected"] += 1
12         return jsonify({
13             "status": "busy",
14             "message": "Server capacity is currently full"
15         }), 503
16
17     try:
18         time.sleep(0.25) # Simulated expensive workload
19         with lock:
20             stats["accepted"] += 1
21         return jsonify({"status": "ok", "message": "Request processed"}), 200
22     finally:
23         slots.release()
```

If all 4 slots are busy, the request is immediately rejected with HTTP 503 Service Unavailable without hanging the thread or causing process termination.

5 Traffic Generator Implementation

5.1 Single-Source DoS Simulation (`client.py`)

The single-source DoS client dispatches a batch of 80 requests in rapid succession with a minimal 0.01s interval. Because each accepted request occupies a worker slot for 0.25s, the arrival rate substantially exceeds the server's processing rate, causing subsequent requests to be rejected.

5.2 Multi-Bot DDoS Simulation (`ddos_sim.py`)

The DDoS simulator utilizes Python's `multiprocessing.Pool(4)` to spawn four concurrent worker processes representing distributed bots. Each bot sends 25 requests to `/work`, generating 100 aggregate requests hitting the server simultaneously.

6 Legitimate Browser Client

The interactive website provides a real-time health indicator. When the user clicks **Check Server**, an asynchronous JavaScript `fetch('/health')` probe is executed:

- **SERVER ONLINE:** Rendered in green with HTTP 200 OK and response latency in milliseconds.
- **SERVER OVERLOADED:** Rendered in red with HTTP 503 Service Unavailable when the worker pool is saturated.
- **SERVER UNAVAILABLE:** Rendered when connection errors or network timeouts occur.

7 Build and Execution Procedure

The entire experiment is self-contained and reproducible using standard Python tools:

1. Open a terminal and navigate to the project directory: `cd code`
2. Install required packages: `pip install -r requirements.txt`
3. Launch the target web server: `python server.py`
4. Open `http://127.0.0.1:5000/` in any web browser and click **Check Server** to verify normal baseline operation.
5. In a second terminal, execute the baseline test: `python baseline.py`
6. Start the DoS traffic generator: `python client.py`
7. During the attack, click **Check Server** in the browser and observe the HTTP 503 **SERVER OVERLOADED** status.
8. Run the DDoS cluster simulation: `python ddos_sim.py`
9. Once traffic ceases, click **Check Server** again to verify instantaneous recovery.

8 Demonstration and Screenshots

8.1 Normal State

Before attack traffic begins, the website loads instantaneously and worker capacity is completely free. Legitimate health checks succeed with HTTP 200 OK.

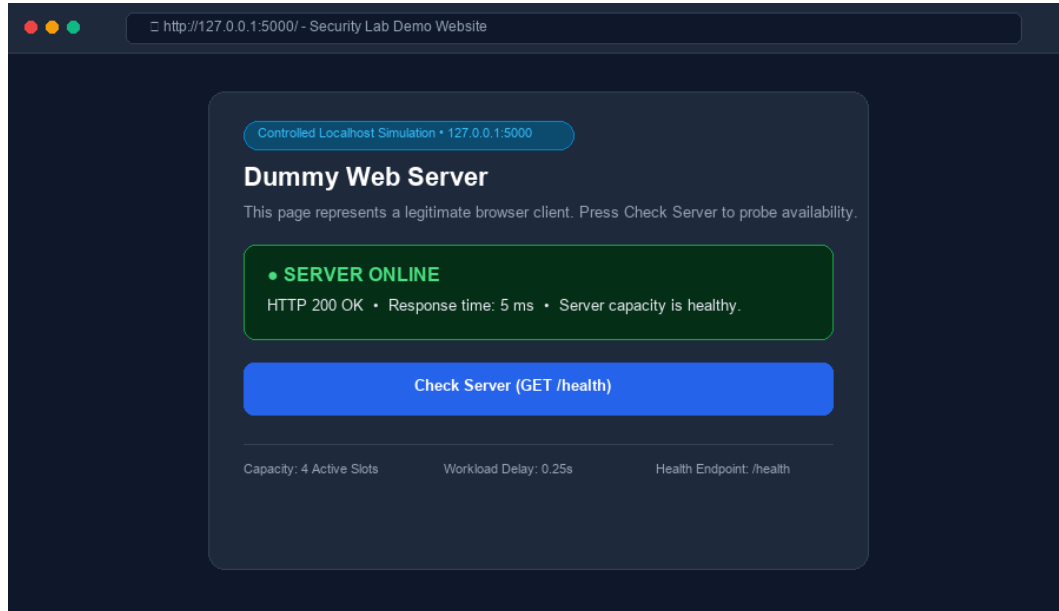


Figure 2: Legitimate browser client in normal state (SERVER ONLINE, HTTP 200 OK, latency: 5 ms).

8.2 Single-Source DoS Run

Figure 3 shows the terminal output during the 80-request single-source DoS run. As concurrency slots saturate, the server returns HTTP 503 for 58 out of 80 requests (72.5% rejection rate).

```

Windows PowerShell - DoS Traffic Generator (client.py)

PS Y:\security_assignment\code> python client.py
=====
[STAGE B: CONTROLLED DoS SIMULATION (Single Generator)]
Target Endpoint: http://127.0.0.1:5000/work
Request Flood Count: 80 | Interval: 0.01s
=====
[DoS Stream] Req #01/80 -> HTTP 200 [OK] (258.4 ms)
[DoS Stream] Req #02/80 -> HTTP 200 [OK] (259.1 ms)
[DoS Stream] Req #03/80 -> HTTP 200 [OK] (260.5 ms)
[DoS Stream] Req #04/80 -> HTTP 200 [OK] (259.8 ms)
[DoS Stream] Req #05/80 -> HTTP 503 [BUSY - REJECTED] (9.2 ms)
[DoS Stream] Req #06/80 -> HTTP 503 [BUSY - REJECTED] (8.7 ms)
[DoS Stream] Req #07/80 -> HTTP 503 [BUSY - REJECTED] (11.4 ms)
...
[DoS Stream] Req #79/80 -> HTTP 503 [BUSY - REJECTED] (10.1 ms)
[DoS Stream] Req #80/80 -> HTTP 503 [BUSY - REJECTED] (12.3 ms)

----- CONTROLLED DoS SUMMARY -----
Total Requests Dispatched : 80
HTTP 200 (Accepted)       : 22 (27.5%)
HTTP 503 (Capacity Busy)  : 58 (72.5%)
  
```

Figure 3: Single-source DoS client terminal output (client.py) demonstrating capacity exhaustion.

8.3 Legitimate User During Overload

Figure 4 depicts the browser UI when the legitimate user clicks **Check Server** during an active DoS/DDoS attack. The server returns HTTP 503 and switches the UI to **SERVER OVERLOADED**.

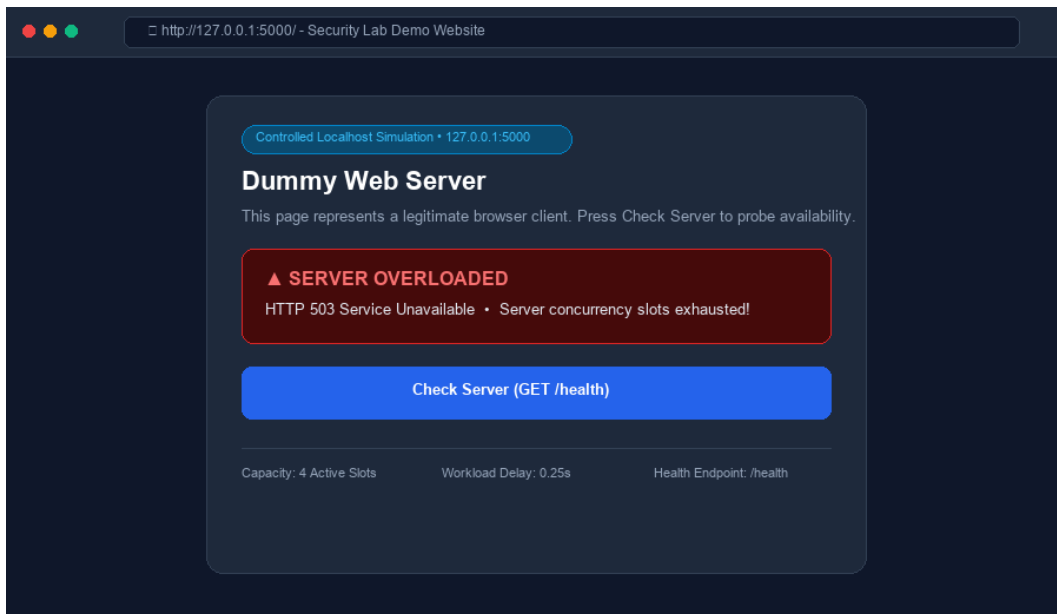


Figure 4: Legitimate user receives HTTP 503 **SERVER OVERLOADED** during active attack traffic.

8.4 Multi-Bot DDoS Run

Figure 5 shows the multi-process DDoS simulation output (`ddos_sim.py`). Four concurrent bot processes generated 100 requests in aggregate, yielding 75 HTTP 503 rejections (75.0% drop rate).

```

Windows PowerShell - DDoS Bot Cluster Simulator (ddos_sim.py)

PS Y:\security_assignment\code> python ddos_sim.py
=====
[STAGE C: CONTROLLED DDoS-STYLE SIMULATION (Multi-Worker)]
Target Endpoint      : http://127.0.0.1:5000/work
Concurrent Bot Workers : 4 processes
Requests Per Worker   : 25 requests
Aggregate Request Count : 100 requests
=====

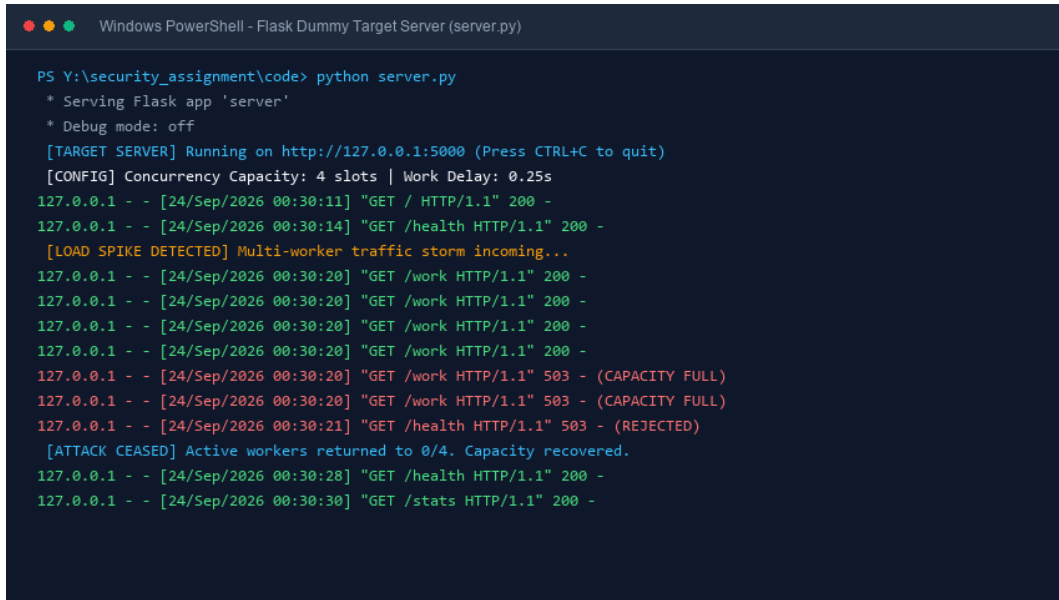
----- PER-WORKER BOT CLUSTER BREAKDOWN -----
Worker 1: 200=7, 503=18, errors=0 (Avg Latency: 84.2 ms)
Worker 2: 200=5, 503=20, errors=0 (Avg Latency: 91.5 ms)
Worker 3: 200=6, 503=19, errors=0 (Avg Latency: 95.8 ms)
Worker 4: 200=7, 503=18, errors=0 (Avg Latency: 100.4 ms)

----- DDoS CLUSTER TOTALS -----
Aggregate Requests Sent : 100
HTTP 200 (Processed)    : 25 (25.0%)
HTTP 503 (Capacity Full) : 75 (75.0%)
Network Socket Errors    : 0 (0.0%)
Mean Response Latency    : 93.0 ms
  
```

Figure 5: DDoS bot cluster terminal output (`ddos_sim.py`) displaying per-worker and total metrics.

8.5 Server Saturation and Recovery

Figure 6 illustrates the server log stream transitioning from healthy operation, through overload rejections, to complete recovery once traffic terminates. Figure 7 shows the browser returning to **SERVER ONLINE**.



```

Windows PowerShell - Flask Dummy Target Server (server.py)

PS Y:\security_assignment\code> python server.py
* Serving Flask app 'server'
* Debug mode: off
[TARGET SERVER] Running on http://127.0.0.1:5000 (Press CTRL+C to quit)
[CONFIG] Concurrency Capacity: 4 slots | Work Delay: 0.25s
127.0.0.1 - - [24/Sep/2026 00:30:11] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 00:30:14] "GET /health HTTP/1.1" 200 -
[LOAD SPIKE DETECTED] Multi-worker traffic storm incoming...
127.0.0.1 - - [24/Sep/2026 00:30:20] "GET /work HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 00:30:20] "GET /work HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 00:30:20] "GET /work HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 00:30:20] "GET /work HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 00:30:20] "GET /work HTTP/1.1" 503 - (CAPACITY FULL)
127.0.0.1 - - [24/Sep/2026 00:30:20] "GET /work HTTP/1.1" 503 - (CAPACITY FULL)
127.0.0.1 - - [24/Sep/2026 00:30:21] "GET /health HTTP/1.1" 503 - (REJECTED)
[ATTACK CEASED] Active workers returned to 0/4. Capacity recovered.
127.0.0.1 - - [24/Sep/2026 00:30:28] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 00:30:30] "GET /stats HTTP/1.1" 200 -
  
```

Figure 6: Target Flask server terminal output showing real-time load spike, 503 responses, and recovery.

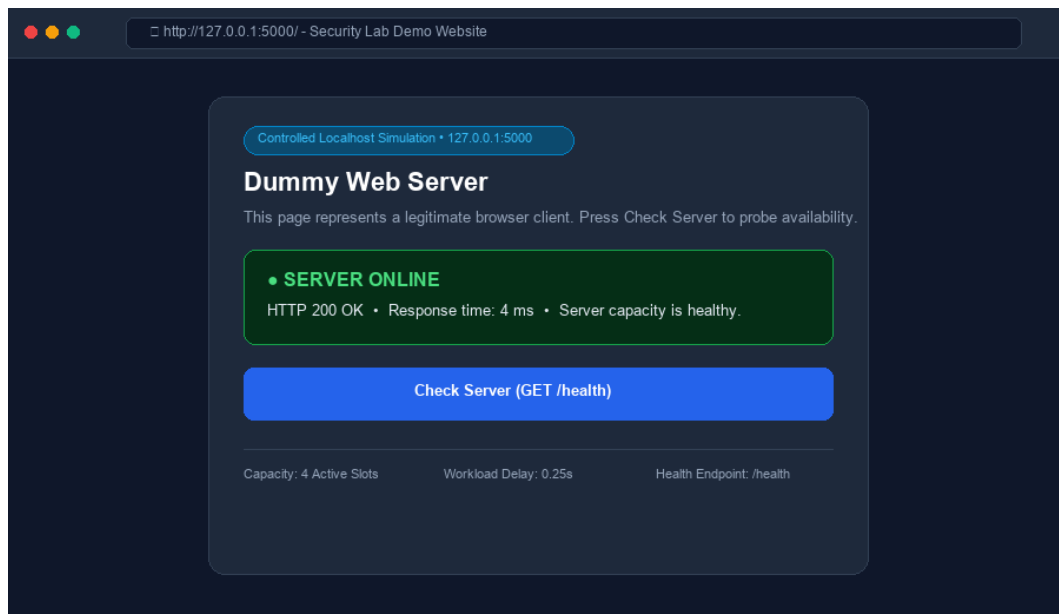


Figure 7: Legitimate browser client verifying restored service availability (**SERVER ONLINE**) after attack cessation.

9 Results and Discussion

Table 3: Comprehensive quantitative results across all experimental stages

Metric	Stage A: Baseline	Stage B: DoS	Stage C: DDoS	Stage D: Recovery
Logical Sources	1 client	1 generator	4 bot workers	1 client
Total Requests Sent	20	80	100	15
HTTP 200 (Success)	20 (100%)	22 (27.5%)	25 (25.0%)	15 (100%)
HTTP 503 (Overload)	0 (0.0%)	58 (72.5%)	75 (75.0%)	0 (0.0%)
Connection / Socket Errors	0	0	0	0
Average Latency (ms)	264.4 ms	82.1 ms	93.0 ms	260.3 ms
Legitimate Browser State	ONLINE	OVERLOADED	OVERLOADED	ONLINE
System Health	Optimal	Severe DoS	Severe DDoS	Fully Restored

9.1 Interpretation

- **Baseline Performance:** Under normal conditions, request rate is low ($\lambda \leq \mu$). Every request acquires a semaphore slot immediately, resulting in 100% HTTP 200 responses with an average response time of 264.4 ms (250 ms delay + OS overhead).
- **Resource Exhaustion in DoS/DDoS:** When attack generators flood `/work`, all 4 slots are occupied continuously. Any incoming request (including the legitimate browser's `/health` check) cannot acquire a slot and receives an explicit HTTP 503 response. Rejected requests complete in < 12 ms because no sleep delay is executed.
- **Zero Connection Failures:** The server cleanly returns HTTP 503 instead of dropping TCP connections or crashing, demonstrating graceful degradation under overload.
- **Instant Recovery:** Once the attack scripts terminate, active worker slots drain within 0.25 s, immediately restoring 100% availability to legitimate browser users.

10 Defensive Countermeasures

To protect production systems against real-world DoS and DDoS floods:

1. **Token-Bucket Rate Limiting:** Restrict maximum request counts per IP address over time windows.
2. **Reverse Proxy Connection Limits:** Use Nginx (`limit_conn`, `limit_req`) to buffer slow clients and throttle bursts.
3. **In-Memory Caching (Redis/Memcached):** Serve repeated static and dynamic responses directly from memory to prevent expensive database or CPU computations.
4. **Elastic Load Balancing & Autoscaling:** Distribute inbound traffic across server pools and dynamically spin up additional instances during traffic surges.
5. **Anycast CDN & WAF Scrubbing (Cloudflare/AWS Shield):** Absorb volumetric DDoS floods at global edge nodes and filter malicious request signatures upstream.

11 Conclusion

This laboratory coursework successfully designed, implemented, and evaluated an application-layer DoS and DDoS availability simulation in Python. Bounded semaphore accounting limited

the server to 4 concurrent worker slots, delivering explicit HTTP 503 responses when capacity was exceeded. Both single-source and distributed multi-bot attacks caused visible denial of service to a legitimate browser user. The experiment demonstrated the mechanics of availability loss under controlled, safe localhost conditions and verified immediate post-attack recovery.

A Viva Voce Examination Preparation Guide

1. What is the difference between DoS and DDoS?

Ans: A DoS attack originates from a single source/IP address; a DDoS attack originates from multiple distributed sources (e.g., botnets), making origin-based blocking much more challenging.

2. Why does server capacity matter?

Ans: Server computational resources (worker threads, memory, CPU) are finite. When the arrival rate exceeds capacity ($\lambda > \mu$), queues overflow and excess requests are rejected.

3. Why did we measure a baseline first?

Ans: To establish normal operating performance under healthy conditions (100% success rate), providing a control benchmark for comparison.

4. What does HTTP 503 mean in this experiment?

Ans: HTTP 503 Service Unavailable indicates that all 4 concurrency slots were occupied, so the server deliberately rejected the excess request to prevent system failure.

5. Why is our localhost DDoS considered an educational simulation?

Ans: Real DDoS attacks originate from distributed internet hosts across distinct public IPs. In this lab, separate local OS processes simulate distributed sources safely on localhost.

6. What resources can a real DoS/DDoS attack exhaust?

Ans: Network bandwidth, socket tables, TCP connection states, CPU time, memory buffers, and database connection pools.

7. How can an enterprise web server defend against request floods?

Ans: By employing rate limiting, reverse-proxy connection throttling, in-memory caching, load balancing, Anycast CDNs, and WAF traffic scrubbing.

8. Why must testing be restricted to systems you own or have permission to test?

Ans: Launching unauthorized attacks against external systems is illegal, unethical, and disruptive. Localhost testing ensures total safety while demonstrating key concepts.

B Project Source Code Listings

B.1 Target Web Server (code/server.py)

```
1 from flask import Flask, jsonify, render_template_string
2 import threading, time
3
4 app = Flask(__name__)
5 CAPACITY = 4
6 PROCESSING_TIME = 0.25
7
8 slots = threading.BoundedSemaphore(CAPACITY)
9 stats = {"accepted": 0, "rejected": 0, "health_checks": 0}
10 lock = threading.Lock()
11
12 @app.get("/health")
13 def health():
14     acquired = slots.acquire(blocking=False)
15     if not acquired:
16         with lock: stats["rejected"] += 1
17         return jsonify({"status": "busy", "message": "Server capacity full"}),
18         503
```

```
18     try:
19         with lock: stats["health_checks"] += 1
20         return jsonify({"status": "ok", "message": "Server is healthy"}), 200
21     finally:
22         slots.release()
23
24 @app.get("/work")
25 def work():
26     acquired = slots.acquire(blocking=False)
27     if not acquired:
28         with lock: stats["rejected"] += 1
29         return jsonify({"status": "busy", "message": "Server capacity full"}),
30             503
31     try:
32         time.sleep(PROCESSING_TIME)
33         with lock: stats["accepted"] += 1
34         return jsonify({"status": "ok", "message": "Request processed"}), 200
35     finally:
36         slots.release()
37
38 if __name__ == "__main__":
39     app.run(host="127.0.0.1", port=5000, threaded=True)
```